Searching (in arrays)

Jarmo Hurri

Overview

- *array* -> fixed lengths sort of
  - fixed-sized container
  - whose elements can be indexed (e.g. `a[0]`)
  - access / modification time *constant* (independent of size)
  - fundamental container in many languages (Java, C++)
- Python has lists instead
- for this discussion array corresponds to
  - list of fixed length so we can't use operations like append as they change the list
  - elements accessed or modified only via indices
  - generating one: e.g. `[0 for _ in range(1000)]`
- we will explore standard operations on arrays
  - search for an element
  - search for element in sorted array
  - efficiency of algorithms
    - time complexity

© Jarmo Hurri

Search

- find whether an element is present in array – definition of search
- a *value* could be attached to the search key
  - this is called a *symbol table*
  - key-value pairs
- examples

application	purpose	key	value
phone catalogue	number of person	name	number
airport security	is passenger on flight	name	–
library	reservations for person	name	book(s)
data security	whitelisting cookie sites	site	–

Sequential (linear) search : simple level

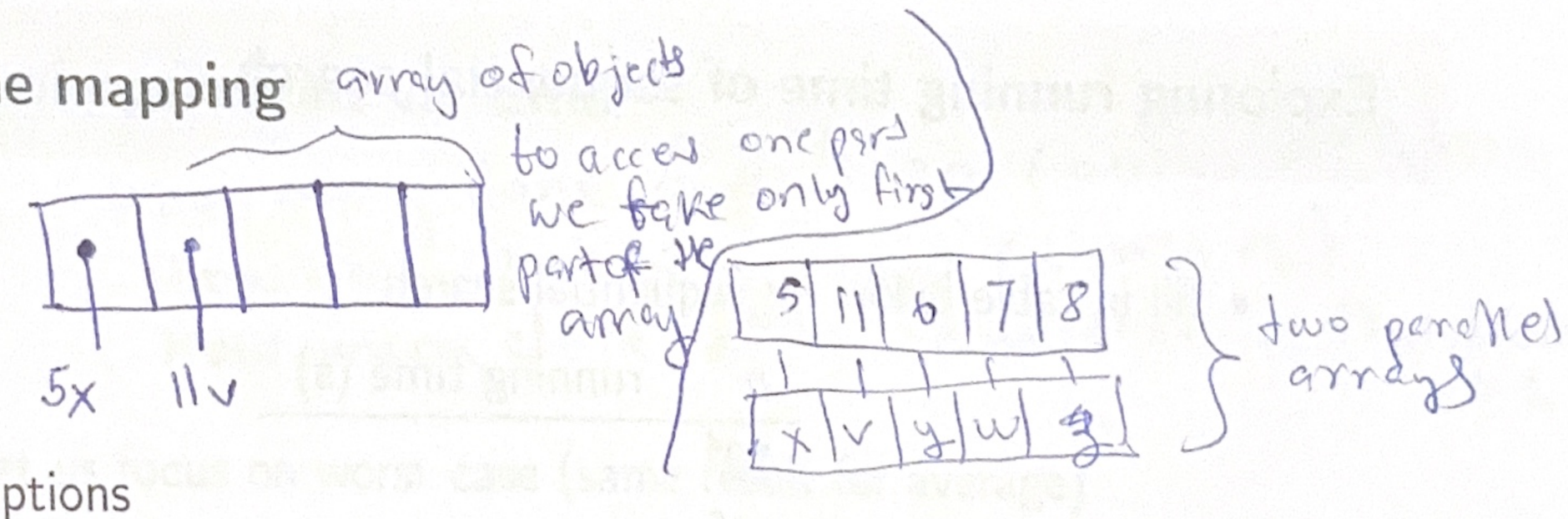
- basic case: find whether element `KEY` is in array `ARRAY` with `N` elements
- pseudocode (one possible formulation)

```
index=0
found=false
while (not found) and (index<N)
      if ARRAY[index]=KEY then
         found=true
      end if
      index=index+1
end loop
output found
```

Sequential search in Python

Write function `seq_search(key, array)` returning `True` if `key` is in list `array` or `False` otherwise.

Key-value mapping



- options
  - two "parallel" arrays, keys and values in matching positions
  - array of objects
- let us write a Python example of parallel arrays, with
  - key: name (string)
  - value: room number (integer)
- then do the same using objects

Measuring running time of search ↪

- wall-clock time used by program depends on other programs
- better measure is computer time (CPU time) used by program
- function `process_time()` in module `time` returns seconds of computer time used by program up to that point

```
def time_search(algo, n, /, sort_data=False):
    import random, time
    # n samples with replacement from [0,1,...,n-1]
    data = random.choices(range(n), k=n)
    if sort_data: data.sort()
    key = n # never found, ensures worst case
    start = time.process_time()
    algo(key, data)
    end = time.process_time()
    return end - start
```


Exploring running time of sequential search

- fill in table below for sequential search

n	running time (s)
10^4	
10^5	
10^6	

- how does running time of sequential search seem to depend on input size n?

- this can in fact be deduced from program code

Number of operations in sequential search

- say each line of code is one operation
- how many operations are executed if array has n elements
  - in worst (slowest) case? n
  - on the average? n/2 or (if just middle value of array)
  - in best (fastest) case? 1
- best case not interesting
- in worst case and on the average, what kind of function is number of operations in terms of n?

linear

Number of operations vs. time

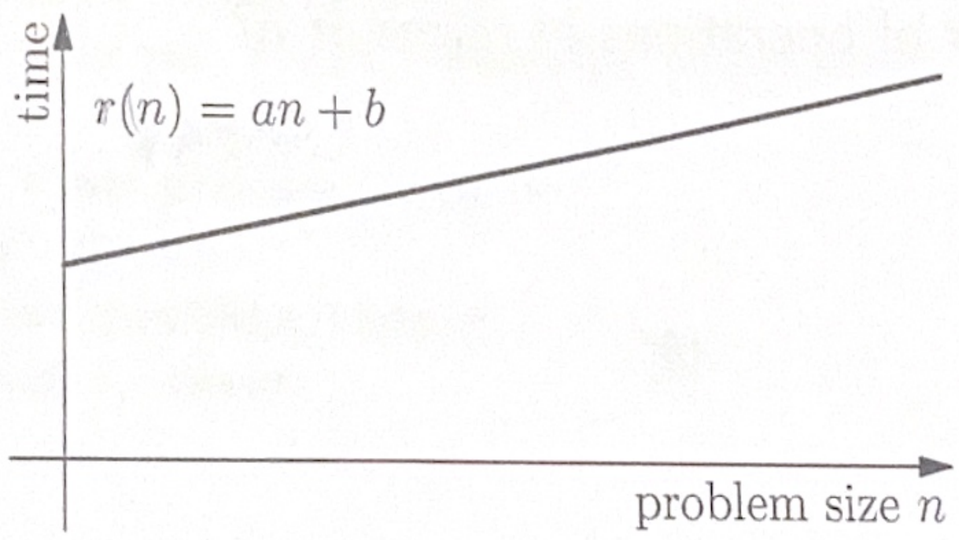
How many done outside loop (only done once)
How many done every loop (times n)
How many done after condition is found (only once + 1)

- let us focus on worst case (same result for average)
- say each operation (line) takes constant time
  - time taken by operation does not depend on n
- what kind of function is time taken in terms of n?

Linear

Big-O notation

- we will adopt *tight-bound* definition of $O(n)$ (often $\Theta(n)$)
- worst case time complexity of algorithm is $O(f(n))$ if, for large values on n, running time is bound
  - from above by $k \times f(n)$, where k is constant
  - from below by $\ell \times f(n)$, where ℓ is constant
- let worst case running time of algorithm be $r(n) = an + b$, where $a > 0$ and $b > 0$ are constants
- why does such algorithm have $O(n)$ worst case complexity?



$\frac{f(n)}{n}$ = constant

$\frac{4n+4}{n} = 4 + \frac{4}{n} \to 0$ as n grows

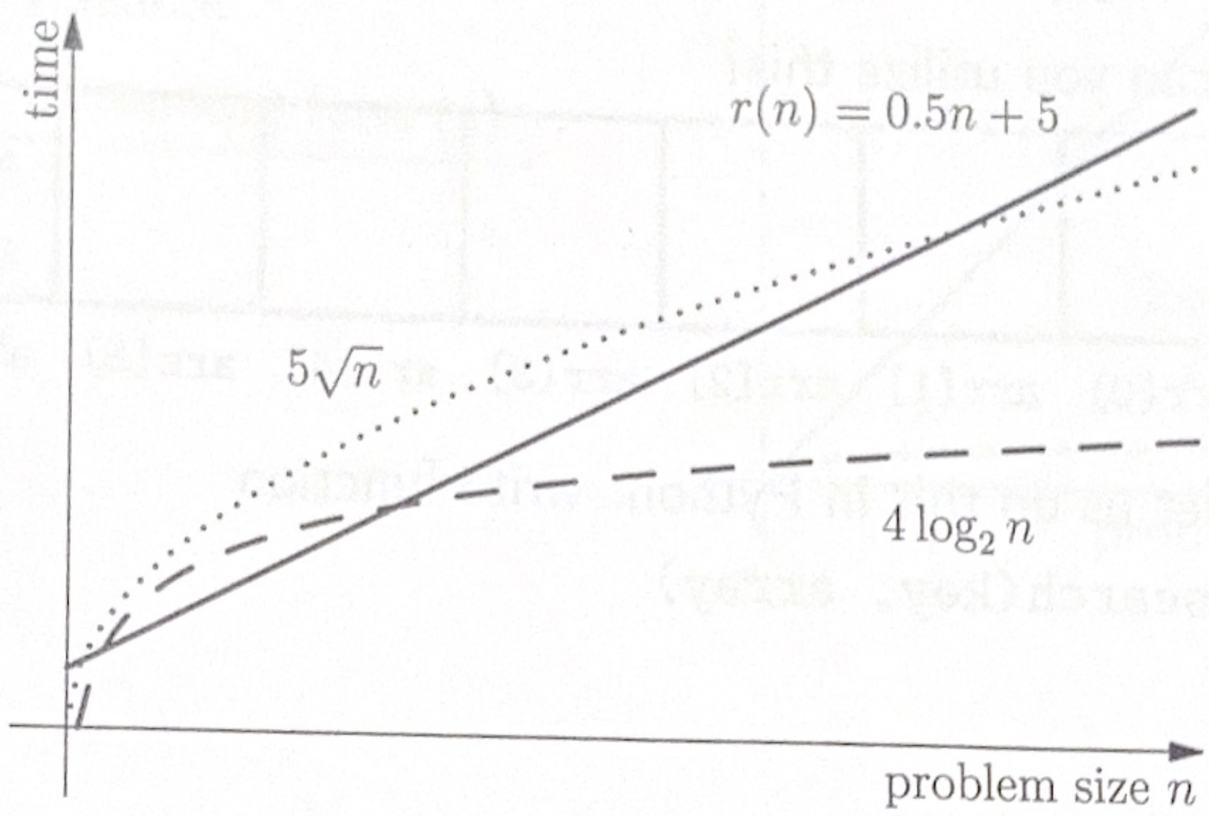
Showing that $an + b$ is $O(n)$

Increasing # of elements by factor of 10 => running time increasing by factor of 10

Given that constants $a > 0$, $b > 0$ and $k > a$, find the values of n for which $kn > an + b$.

How tight a bound is $O(n)$?

- $O(n)$ gives tight bounds on sequential search
- can it be improved? no as the others flatten
- is $an + b$ in
  - $O(\sqrt{n})$?
  - $O(\log_2 n)$?



Predicting increase of running time of $O(n)$

- $O(n)$ implies: for large values of n, running time depends linearly on input size
- so what would doubling input size predict for running time?

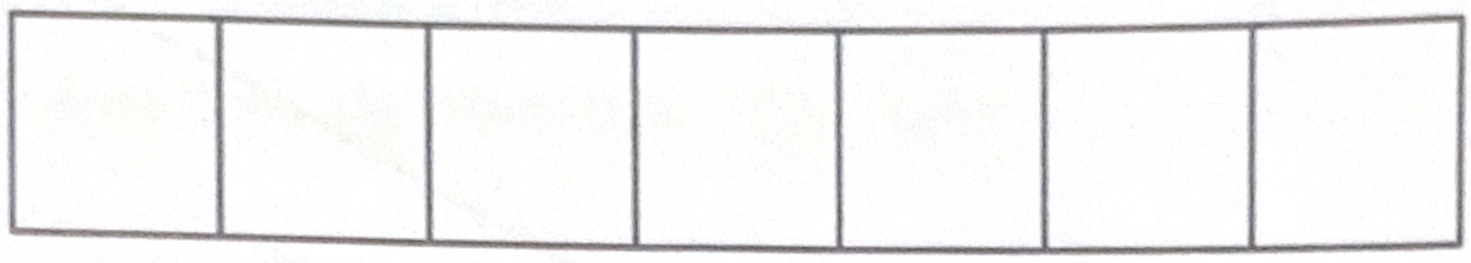
running time also doubles

mid = lo + high / 2

upper boundary denoted by high
lower boundary denoted by low
go to middle point (n/2)
then figure out if you go left or right

Binary search

- problem same: looking for key in array
- but now assume your array is *sorted* so that elements are in increasing (or decreasing) order
- how can you utilize this?



- now let us do this in Python: write function `bin_search(key, array)`

Time complexity of binary search

- worst case: array size n is halved until only one is left
- let the number of loop rounds needed for this be x
- then

steps	remaining
0	n
1	$\frac{n}{2}$
2	$\frac{n}{4}$
3	$\frac{n}{8}$
x	$\frac{n}{2^x}$

worst case

$\frac{n}{2^x} = 1$

$n = 2^x$

$x = \log_2 n$

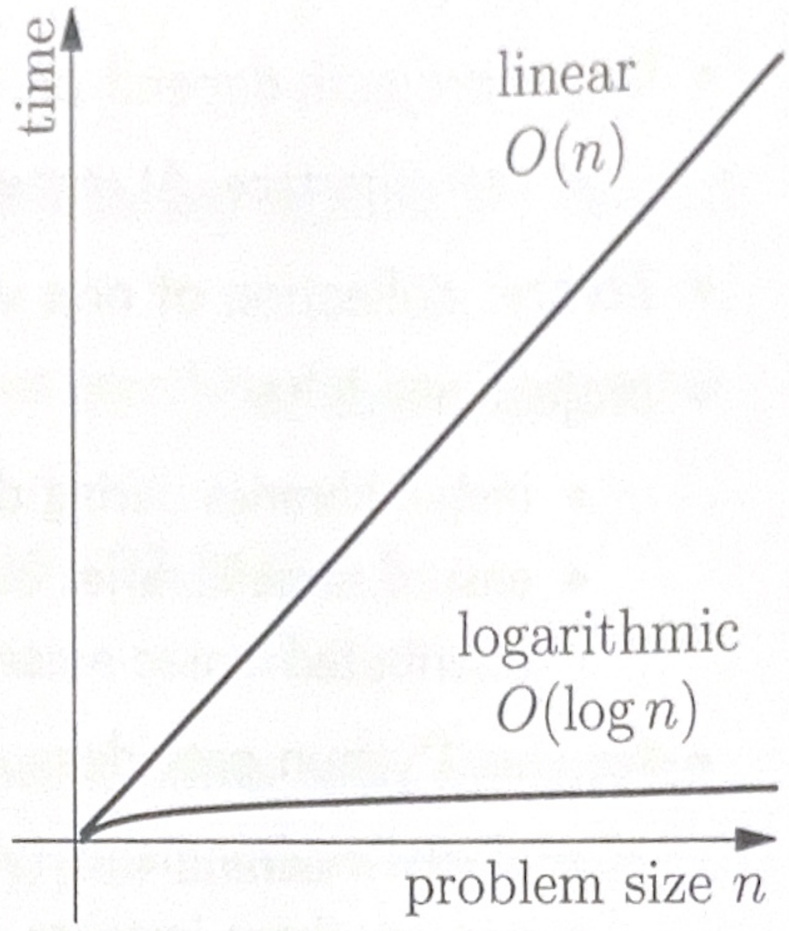
- time complexity of binary search is

Logarithmic against linear order of growth

running time increases by 1 only so not proportional same increase in data

- fill in table below for binary search

n	running time (s)
10^4	
10^5	
10^6	



Asymptotic time complexity and order of growth

- in loop body, binary search does more than sequential
  - say one body evaluation takes "100 times longer"
- so comparing running times means comparing (graphically)

$\log_2 x$ and x so $\nearrow$ liner

$\searrow$

outperforms liner

- *asymptotic*: examining running times of algorithms when problem size n becomes very large
- in these conditions, *order of growth* of most frequent operations is what typically matters
  - leading coefficient is secondary
- depends only on *most significant term*
  - example: graphically compare n^3 and $n^3 + 10n^2$ for large values of n

this is going to be insignificant for large values

Using external libraries in Python

- larger projects depend on external code
  - user interface, AI, networking...
- *library*: collection of non-volatile resources (files)
- project needs to
  - install libraries during development
  - ensure same libraries can be installed when project is distributed
- for this Python provides
  - virtual environments (`venv`): install libraries inside project
  - `pip`: package installer

Using `venv` and `pip` in VS code (package `requests`)

1. select `git bash` as default terminal profile
   - `F1` terminal profile
2. initialize new git project
   - `F1` git init
3. create `venv` for this project
   - `F1` python create env
4. open terminal
   - `ctrl-shift-‘`
5. run pip to install `requests`

   ```
   python -m pip install requests
   ```

6. record info on used packages in project file `requirements.txt`

   ```
   python -m pip freeze > requirements.txt
   ```

Homework 1

1. How many comparisons of the search key against an element of the array does
   1.1 sequential search perform when searching for key 15 in array containing elements

   `-2, 9, 11, 12, 13, 14, 15, 22, 142`

   1.2 binary search perform in the same search?
2. Modify binary search code so that it prints each element it compares the key against. Verify your answer from above.

Homework 2

Use sequential search in Python to find the correct grade for student `Charlie` given the two parallel arrays below.

```
students = ['Alissa', 'Ben', 'Charlie', 'Dianna']
grades = ['B', 'D', 'B', 'A']
```

Homework 3

Why is the following statement true?

Array element access and array element modification both have worst-case time complexity $O(1)$.

meens that it taks the same amount of time to run worst case as compered to best case.

amount of time data takes to go through certin set of data

Homework 4

You have one minute of computing time, and your computer performs 10^9 operations per second. Find the maximum input size n of a problem solved in this time, given that the number of operations to solve the problem is

1. $\log_2 n$
2. n
3. n^2
4. 2^n.

Homework 5

The repository `https://github.com/dwyl/english-words` contains file `words_alpha.txt` with over 370000 English words, lowercase, in alphabetical order. These were obtained from various online resources. Because of the source of these words, not all of them are "proper" words included in an English dictionary.

The function below can be used to download all the words given the URL address: ↪

```
def read_words_from_url(url):
    import requests

    response = requests.get(url, stream=True)
    words = [line.decode('utf-8').strip() for line in response.iter_lines()]

    return words
```


© Jarmo Hurri

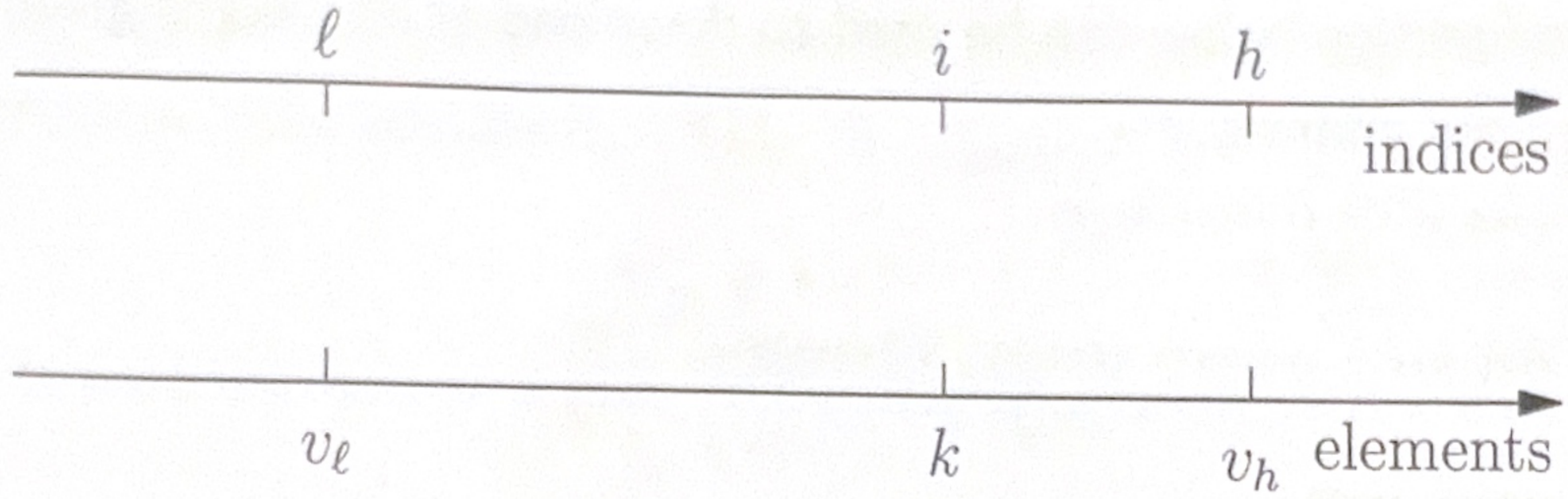
Homework continues

1. Download the words and print the last 10. To get a URL address of the file, click on the file, then select `Raw`, and finally copy the URL address from the address line of your browser.
2. Apply sequential search to the data. Select your own search keys.
3. Apply binary search to the data.
4. Run an experiment to compare the running time of sequential search and binary search for this data.

© Jarmo Hurri

Homework 6

Interpolation search is a variant of binary search. Assume the array is in increasing order. In interpolation search, the next search index i is at the same relative position between lo (ℓ) and hi (h) indices as the key (k) is between element v_ℓ at index ℓ and element v_h at index h.



© Jarmo Hurri

Homework continues

1. Use the diagram above to deduce that

$$i = \ell + \frac{k - v_\ell}{v_h - v_\ell} \times (h - \ell).$$

2. Binary search guarantees that for the next search index i we always have $\ell \leq i \leq h$. This is not the case for the formula above.
   2.1 Find i if $k = 200$, $v_\ell = 0$, $v_h = 100$, $\ell = 0$ and $h = 1$.
   2.2 What should the result of the algorithm be if $k < v_\ell$ or $k > v_h$.
3. To avoid division by zero, what should be the value of i if $v_h = v_\ell$?
4. Implement interpolation search.

© Jarmo Hurri